



UNIVERSIDAD AUTÓNOMA DE
CHIHUAHUA



Proyecto 1a. Evaluación

**Universidad Autónoma de Chihuahua
Facultad de Ingeniería**

**Materia y grupo: Bases de datos avanzadas
5K2**

Docente: Jose Saul De Lira Miramontes

Equipo:

Edgar Alejandro Gutierrez Carrillo 385655

Emilio Gallardo Medrano 385530

Carlos Alberto Diaz Sanchez 385539

Introducción.

Este reporte documenta el modelado y manipulación de la base de datos NoSQL **BDAutos** en MongoDB. Esta base de datos aplica dos patrones fundamentales: Embedding (anidamiento del catálogo de modelos dentro del fabricante) y Linking (referenciación de cada automóvil hacia su fabricante mediante el makerId). El documento detalla la resolución de 25 requerimientos analíticos resueltos a través del Aggregation Framework.

Script de la Base de Datos.

```
db.makers.insertMany([
  {
    _id: 1,
    name: "Volkswagen",
    fullName: "Volkswagen AG",
    country: "Germany",
    continent: "Europe",
    models: [{ name: "Jetta" }, { name: "Golf" }]
  },
  {
    _id: 2,
    name: "Toyota",
    fullName: "Toyota Motor Corporation",
    country: "Japan",
    continent: "Asia",
    models: [{ name: "Corolla" }, { name: "Prius" }]
  },
  {
    _id: 3,
    name: "Ford",
    fullName: "Ford Motor Company",
    country: "USA",
    continent: "America",
    models: [{ name: "Mustang" }, { name: "F-150" }]
  },
  {
    _id: 4,
    name: "Honda",
    fullName: "Honda Motor Co.",
    country: "Japan",
    continent: "Asia",
    models: [{ name: "Civic" }, { name: "Accord" }]
  },
])
```

```

{
  _id: 5,
  name: "Nissan",
  fullName: "Nissan Motor Co.",
  country: "Japan",
  continent: "Asia",
  models: [{ name: "Sentra" }, { name: "GTR" }, { name: "Altima" }]
},
{
  _id: 6,
  name: "Audi",
  fullName: "Audi AG",
  country: "Germany",
  continent: "Europe",
  models: [{ name: "A4" }]
}
]);

```

```

db.cars.insertMany([
  { _id: 101, makerId: 1, model_name: "Jetta", description: "Jetta 2.0", year:
2024, cylinders: 4, hp: 150, weight_lb: 3000, accel_0_60_s: 8.5, mpg: 36 },
  { _id: 102, makerId: 1, model_name: "Golf", description: "Golf R 2.0t",
year: 2024, cylinders: 4, hp: 315, weight_lb: 3300, accel_0_60_s: 4.5, mpg: 28
},
  { _id: 103, makerId: 2, model_name: "Prius", description: "Prius Hybrid",
year: 2024, cylinders: 4, hp: 120, weight_lb: 2900, accel_0_60_s: 9.8, mpg: 55
},
  { _id: 104, makerId: 3, model_name: "Mustang", description: "Mustang GT",
year: 2023, cylinders: 8, hp: 450, weight_lb: 3800, accel_0_60_s: 4.2, mpg: 19
},
  { _id: 105, makerId: 3, model_name: "F-150", description: "F-150 Raptor",
year: 2022, cylinders: 8, hp: 400, weight_lb: 5000, accel_0_60_s: 5.5, mpg: 15
},
  { _id: 106, makerId: 4, model_name: "Civic", description: "Civic Si 1.5t",
year: 2023, cylinders: 4, hp: 200, weight_lb: 2900, accel_0_60_s: 6.8, mpg: 38
},
  { _id: 107, makerId: 4, model_name: "Accord", description: "Accord V6",
year: 2018, cylinders: 6, hp: 278, weight_lb: 3400, accel_0_60_s: 5.8, mpg: 25
},
  { _id: 108, makerId: 5, model_name: "GTR", description: "GTR 3.8t", year:
2024, cylinders: 6, hp: 565, weight_lb: 3900, accel_0_60_s: 2.9, mpg: 16 }
]);

```

Consultas y Resultados.

- 1. Obtener los autos de modelo 2024 en adelante con rendimiento de al menos 35 mpg. Mostrar sólo la descripción, el año y el rendimiento, ordenados de mayor a menor rendimiento.

```
db.cars.find(  
  { year: { $gte: 2024 }, mpg: { $gte: 35 } },  
  { description: 1, year: 1, mpg: 1, _id: 0 }  
) .sort({ mpg: -1 })
```

```
{ description: 'Prius Hybrid', year: 2024, mpg: 55 },  
{ description: 'Jetta 2.0', year: 2024, mpg: 36 }
```

- 2. Listar los autos de fabricantes europeos con motor de 4 cilindros y al menos 250 hp, mostrando descripción, país y potencia, ordenados por potencia descendente.

```
db.cars.aggregate([  
  { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },  
  { $unwind: "$maker" },  
  { $match: { "maker.continent": "Europe", cylinders: 4, hp: { $gte: 250 } } },  
  { $project: { _id: 0, description: 1, country: "$maker.country", hp: 1 } },  
  { $sort: { hp: -1 } }  
])
```

```
{ description: 'Golf R 2.0t', hp: 315, country: 'Germany' } ]
```

- 3. Contar cuántos autos de fabricantes japoneses tienen motor turbo, es decir, cuya descripción termina con la cilindrada seguida de "t" (por ejemplo "1.5t").

```
db.cars.aggregate([  
  { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },  
  { $match: { "maker.country": "Japan", description: /t$/i } },  
  { $count: "autos_turbo_japoneses" }  
])
```

```
[ { autos_turbo_japoneses: 2 } ]
```

- 4. Encontrar los autos que cumplan cualquiera de estas condiciones: (a) pesan menos de 3000 lb y aceleran de 0 a 60 mph en menos de 9 s, o (b) tienen 8 o más cilindros y rinden al menos 19 mpg.

```
db.cars.find({
  $or: [
    { weight_lb: { $lt: 3000 }, accel_0_60_s: { $lt: 9 } },
    { cylinders: { $gte: 8 }, mpg: { $gte: 19 } }
  ]
})
```

```
[
  {
    _id: 104,
    makerId: 3,
    model_name: 'Mustang',
    description: 'Mustang GT',
    year: 2023,
    cylinders: 8,
    hp: 450,
    weight_lb: 3800,
    accel_0_60_s: 4.2,
    mpg: 19
  },
  {
    _id: 106,
    makerId: 4,
    model_name: 'Civic',
    description: 'Civic Si 1.5t',
    year: 2023,
    cylinders: 4,
    hp: 200,
    weight_lb: 2900,
    accel_0_60_s: 6.8,
    mpg: 38
  }
]
```

- 5. Obtener los fabricantes que comercializan más de una marca o línea de modelo, mostrando nombre, país y modelos.

```
db.makers.find(
  { "models.1": { $exists: true } },
  { _id: 0, name: 1, country: 1, models: 1 }
)
```

```
{
  name: 'Volkswagen',
  country: 'Germany',
  models: [ { name: 'Jetta' }, { name: 'Golf' } ]
},
{
  name: 'Toyota',
  country: 'Japan',
  models: [ { name: 'Corolla' }, { name: 'Prius' } ]
},
{
  name: 'Ford',
  country: 'USA',
  models: [ { name: 'Mustang' }, { name: 'F-150' } ]
},
{
  name: 'Honda',
  country: 'Japan',
  models: [ { name: 'Civic' }, { name: 'Accord' } ]
},
{
  name: 'Nissan',
  country: 'Japan',
  models: [ { name: 'Sentra' }, { name: 'GTR' }, { name: 'Altima' } ]
}
```

- 6. Encontrar los autos cuya marca (model.name) es distinta del nombre de su grupo fabricante (maker.name). Mostrar descripción, marca y fabricante.

```
db.cars.aggregate([
  { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as:
"maker" } },
  { $unwind: "$maker" },
  { $match: { $expr: { $ne: ["$model_name", "$maker.name"] } } },
  { $project: { _id: 0, description: 1, marca: "$model_name", fabricante:
"$maker.name" } }
])
```

```
{ description: 'Jetta 2.0', marca: 'Jetta', fabricante: 'Volkswagen' },
{ description: 'Golf R 2.0t', marca: 'Golf', fabricante: 'Volkswagen' },
{ description: 'Prius Hybrid', marca: 'Prius', fabricante: 'Toyota' },
{ description: 'Mustang GT', marca: 'Mustang', fabricante: 'Ford' },
{ description: 'F-150 Raptor', marca: 'F-150', fabricante: 'Ford' },
{ description: 'Civic Si 1.5t', marca: 'Civic', fabricante: 'Honda' },
{ description: 'Accord V6', marca: 'Accord', fabricante: 'Honda' },
{ description: 'GTR 3.8t', marca: 'GTR', fabricante: 'Nissan' }
```

- 7. Calcular por país el número de autos, el rendimiento promedio y la potencia promedio (redondeados a 1 decimal), ordenado por número de autos.

```
db.cars.aggregate([
  { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as:
"maker" } },
  { $unwind: "$maker" },
  { $group: {
    _id: "$maker.country",
    numAutos: { $sum: 1 },
    avgMpg: { $avg: "$mpg" },
    avgHp: { $avg: "$hp" }
  }},
  { $project: { numAutos: 1, avgMpg: { $round: ["$avgMpg", 1] }, avgHp: {
$round: ["$avgHp", 1] } } },
  { $sort: { numAutos: -1 } }
])
```

```
{ _id: 'Japan', numAutos: 4, avgMpg: 33.5, avgHp: 290.8 },
{ _id: 'Germany', numAutos: 2, avgMpg: 32, avgHp: 232.5 },
{ _id: 'USA', numAutos: 2, avgMpg: 17, avgHp: 425 }
```

- 8. Calcular para cada grupo fabricante el número de autos, el número de marcas distintas con autos registrados y la potencia promedio (redondeada a 1 decimal), ordenado por número de autos.

```
db.cars.aggregate([
  { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as:
"maker" } },
  { $unwind: "$maker" },
  { $group: {
    _id: "$maker.name",
    numAutos: { $sum: 1 },
    marcasDistintas: { $addToSet: "$model_name" },
    avgHp: { $avg: "$hp" }
  }},
  { $project: { numAutos: 1, numMarcasDistintas: { $size: "$marcasDistintas"
}, avgHp: { $round: ["$avgHp", 1] } } },
  { $sort: { numAutos: -1 } }
])
```

```
{ _id: 'Volkswagen', numAutos: 2, numMarcasDistintas: 2, avgHp: 232.5 },
{ _id: 'Ford', numAutos: 2, numMarcasDistintas: 2, avgHp: 425 },
{ _id: 'Honda', numAutos: 2, numMarcasDistintas: 2, avgHp: 239 },
{ _id: 'Nissan', numAutos: 1, numMarcasDistintas: 1, avgHp: 565 },
{ _id: 'Toyota', numAutos: 1, numMarcasDistintas: 1, avgHp: 120 }
```

- 9. Contar cuántos modelos (marcas) hay por continente

```
db.makers.aggregate([
  { $unwind: "$models" },
  {
    $group: {
      _id: "$continent",
      total_modelos: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      continente: "$_id",
      total_modelos: 1
    }
  }
]);
```

```
{ total_modelos: 2, continente: 'America' },
{ total_modelos: 3, continente: 'Europe' },
{ total_modelos: 7, continente: 'Asia' }
```

- 10. Clasificar los autos en rangos de rendimiento [0,20), [20,25), [25,30), [30,35), [35,40) y 40 o más, contando cuántos hay en cada rango y su peso promedio.

```
db.cars.aggregate([
{
  $bucket: {
    groupBy: "$mpg",
    boundaries: [0, 20, 25, 30, 35, 40],
    default: "40 o mas",
    output: {
      total_autos: { $sum: 1},
      peso_promedio: { $avg: "$weight_lb" }
    }
  }
},
{
  $project: {
    _id: 0,
    rango_mpg: {
      $switch: {
        branches: [
          { case: { $eq: ["$_id", 0]}, then: "[0,20)" },
          { case: { $eq: ["$_id", 20]}, then: "[20,25)" },
          { case: { $eq: ["$_id", 25]}, then: "[25,30)" },
          { case: { $eq: ["$_id", 30]}, then: "[30,35)" },
          { case: { $eq: ["$_id", 35]}, then: "[35,40)" }
        ],
        default: "40 o mas"
      }
    },
    total_autos: 1,
    peso_promedio: { $round: ["$peso_promedio", 1] }
  }
}
]);
```

```
{ total_autos: 3, rango_mpg: '[0,20)', peso_promedio: 4233.3 },
{ total_autos: 3, rango_mpg: '[25,30)', peso_promedio: 3293.3 },
{ total_autos: 2, rango_mpg: '[35,40)', peso_promedio: 2950 },
{ total_autos: 1, rango_mpg: '40 o mas', peso_promedio: 2900 }
```

- 11. Obtener los 5 fabricantes con mayor rendimiento promedio, considerando solo aquellos con al menos 8 autos registrados.

```
db.cars.aggregate([
{
  $group: {
    _id: "$makerId",
```

```

        total_autos: { $sum: 1},
        rendimiento_promedio: {$avg: "$mpg"}
    }
},
{
    $match: {
        total_autos: {$gte: 8}
    }
},
{
    $lookup: {
        from: "makers",
        localField: "_id",
        foreignField: "_id",
        as: "maker"
    }
},
{ $unwind: "$maker"},
{
    $project: {
        _id: 0,
        fabricante: "$maker.name",
        total_autos: 1,
        rendimiento_promedio: {$round: ["$rendimiento_promedio",2]}
    }
},
{$sort: {rendimiento_promedio: -1}},
{$limit: 5}
]);

```

ya que ninguno de nuestros fabricantes tiene al menos 8 autos registrados el filtro \$match descarta todos los fabricantes y devuelve una lista vacía

- **12. Clasificar cada auto como "Alta" ($\text{mpg} \geq 30$), "Media" ($20 \leq \text{mpg} < 30$) o "Baja" ($\text{mpg} < 20$) y contar cuántos autos hay por continente y categoría.**

```

db.cars.aggregate([
    {
        $lookup: {
            from: "makers",
            localField: "makerId",
            foreignField: "_id",
            as: "maker"
        }
    },
    { $unwind: "$maker" },
    {
        $project:{
            continente: "$maker.continent",

```

```

        categoria: {
          $switch: {
            branches: [
              {case: {$gte: ["$mpg", 30]}, then: "Alta"},
              {case: {$gte: ["$mpg", 20]}, then: "Media"}
            ],
            default: "Baja"
          }
        }
      },
    },
    {
      $group: {
        _id: {
          continente: "$continente",
          categoria: "$categoria"
        },
        total_autos: { $sum: 1 }
      }
    },
    {
      $project: {
        _id: 0,
        continente: "$_id.continente",
        categoria: "$_id.categoria",
        total_autos: 1
      }
    },
    { $sort : {continente: 1, categoria: 1}}
  ]);

```

```

{ total_autos: 2, continente: 'America', categoria: 'Baja' },
{ total_autos: 2, continente: 'Asia', categoria: 'Alta' },
{ total_autos: 1, continente: 'Asia', categoria: 'Baja' },
{ total_autos: 1, continente: 'Asia', categoria: 'Media' },
{ total_autos: 1, continente: 'Europe', categoria: 'Alta' },
{ total_autos: 1, continente: 'Europe', categoria: 'Media' }

```

- 13. Desde la colección makers, obtener cuántos autos tiene registrados cada fabricante (incluidos los que tienen cero)

```

db.makers.aggregate([
{
  $lookup: {
    from: "cars",
    localField: "_id",
    foreignField: "makerId",
    as: "autos"
  }
},

```

```
{
  $project: {
    _id: 0,
    fabricante: "$name",
    pais: "$country",
    total_autos: {$size: "$autos" }
  }
},
{ $sort: { total_autos: -1}}
]);
```

```
{ fabricante: 'Volkswagen', pais: 'Germany', total_autos: 2 },
{ fabricante: 'Ford', pais: 'USA', total_autos: 2 },
{ fabricante: 'Honda', pais: 'Japan', total_autos: 2 },
{ fabricante: 'Toyota', pais: 'Japan', total_autos: 1 },
{ fabricante: 'Nissan', pais: 'Japan', total_autos: 1 },
{ fabricante: 'Audi', pais: 'Germany', total_autos: 0 }
```

- **14. Encontrar los modelos del catálogo que no tienen ningún auto registrado**

```
db.makers.aggregate([
  { $unwind: "$models"},
  {
    $lookup: {
      from: "cars",
      localField: "models.name",
      foreignField: "model_name",
      as: "autos_registrados"
    }
  },
  {
    $match: {
      autos_registrados: {$size: 0}
    }
  },
  {
    $project: {
      _id: 0,
      fabricante: "$name",
      pais: "$country",
      modelo_sin_registro: "$models.name"
    }
  },
  { $sort: { fabricante: 1, modelo_sin_registro: 1}}
]);
```

```

{ fabricante: 'Audi', pais: 'Germany', modelo_sin_registro: 'A4' },
{ fabricante: 'Nissan', pais: 'Japan', modelo_sin_registro: 'Altima' },
{ fabricante: 'Nissan', pais: 'Japan', modelo_sin_registro: 'Sentra' },
{ fabricante: 'Toyota', pais: 'Japan', modelo_sin_registro: 'Corolla' }
]

```

- **15. Para los 5 autos más potentes, obtener su descripción, potencia y el nombre completo (fullName) de su fabricante**

```

db.cars.aggregate([
{
  $sort: {
    hp: -1
  }
},
{
  $limit: 5
},
{
  $lookup: {
    from: "makers",
    localField: "makerId",
    foreignField: "_id",
    as: "maker"
  }
},
{
  $unwind: "$maker"
},
{
  $project: {
    _id: 0,
    descripcion: "$description",
    potencia: "$hp",
    fabricante: "$maker.fullName"
  }
}
]);

```

```

[
  {
    descripcion: 'GTR 3.8t',
    potencia: 565,
    fabricante: 'Nissan Motor Co.'
  },
  {
    descripcion: 'Mustang GT',
    potencia: 450,
    fabricante: 'Ford Motor Company'
  },
  {
    descripcion: 'F-150 Raptor',
    potencia: 400,
    fabricante: 'Ford Motor Company'
  },
  {
    descripcion: 'Golf R 2.0t',
    potencia: 315,
    fabricante: 'Volkswagen AG'
  },
  {
    descripcion: 'Accord V6',
    potencia: 278,
    fabricante: 'Honda Motor Co.'
  }
]

```

- 16. Obtener, para cada continente, los 3 autos con mayor rendimiento (descripción y mpg).

```

db.cars.aggregate([
  {
    $lookup: {
      from: "makers",
      localField: "makerId",
      foreignField: "_id",
      as: "maker"
    }
  },
  {
    $unwind: "$maker"
  },
  {
    $setWindowFields: {
      partitionBy: "$maker.continent",
      sortBy: {mpg: -1},
      output: {
        posicion: {
          $documentNumber: {}
        }
      }
    }
  },
  {
    $match: {
      posicion: { $lte: 3 }
    }
  },
  {
    $project: {
      _id: 0,

```

```

        continente: "$maker.continente",
        descripcion: "$description",
        mpg: 1
    }
},
{
    $sort: { continente: 1, mpg: 1}
}
]);

```

```

[
  { mpg: 15, continente: 'America', descripcion: 'F-150 Raptor' },
  { mpg: 19, continente: 'America', descripcion: 'Mustang GT' },
  { mpg: 25, continente: 'Asia', descripcion: 'Accord V6' },
  { mpg: 38, continente: 'Asia', descripcion: 'Civic Si 1.5t' },
  { mpg: 55, continente: 'Asia', descripcion: 'Prius Hybrid' },
  { mpg: 28, continente: 'Europe', descripcion: 'Golf R 2.0t' },
  { mpg: 36, continente: 'Europe', descripcion: 'Jetta 2.0' }
]

```

- 17. Para cada fabricante con más de un modelo, determinar cuál de sus modelos tiene más autos registrados.

```

db.makers.aggregate([
  { $match: {"models.1": { $exists: true }}},
  {
    $lookup: {
      from: "cars",
      localField: "_id",
      foreignField: "makerId",
      as: "autos"
    }
  },
  { $unwind: "$autos"},
  {
    $group: {
      _id: {maker: "$name", modelo: "$autos.model_name"},
      count: { $sum: 1}}
  },
  { $sort: {count: -1}
  },
  {
    $group: {
      _id: "$_id.maker",
      modeloTop: {$first: "$_id.modelo"},
      numAutos: {$first: "$count"}
    }
  }
]);

```

```
{ _id: 'Nissan', modeloTop: 'GTR', numAutos: 1 },
{ _id: 'Ford', modeloTop: 'Mustang', numAutos: 1 },
{ _id: 'Honda', modeloTop: 'Accord', numAutos: 1 },
{ _id: 'Volkswagen', modeloTop: 'Golf', numAutos: 1 },
{ _id: 'Toyota', modeloTop: 'Prius', numAutos: 1 }
```

- 18. Calcular el porcentaje que representa cada continente sobre el total de autos, con 2 decimales.

```
PS C:\Users\elsi_> db.cars.aggregate([
>> { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },
>> { $unwind: "$maker" },
>> { $group: { _id: "$maker.continente", total_autos: { $sum: 1 } } },
>> {
>>   $group: {
>>     _id: null,
>>     continentes: { $push: { continente: "$_id", total_autos: "$total_autos" } },
>>     gran_total: { $sum: "$total_autos" }
>>   }
>> },
>> { $unwind: "$continentes" },
>> {
>>   $project: {
>>     _id: 0,
>>     continente: "$continentes.continente",
>>     total_autos: "$continentes.total_autos",
>>     porcentaje: {
>>       $round: [
>>         { $multiply: [{ $divide: ["$continentes.total_autos", "$gran_total"] }, 100] },
>>         2
>>       ]
>>     }
>>   }
>> },
>> { $sort: { porcentaje: -1 } }

>> { "continente": "Asia", "total_autos": 4, "porcentaje": 50 },
>> { "continente": "Europe", "total_autos": 2, "porcentaje": 25 },
>> { "continente": "America", "total_autos": 2, "porcentaje": 25 }
```

- 19. Listar los 5 autos que más superan el rendimiento promedio de los autos con su mismo número de cilindros, mostrando ese promedio y la diferencia.

```
PS C:\Users\elsi_> db.cars.aggregate([
>> {
>>   $setWindowFields: {
>>     partitionBy: "$cylinders",
>>     output: {
>>       promedio_cilindros: { $avg: "$mpg" }
>>     }
>>   }
>> },
>> {
>>   $project: {
>>     _id: 0,
>>     descripcion: "$description",
>>     cilindros: "$cylinders",
>>     mpg: 1,
>>     promedio_cilindros: { $round: ["$promedio_cilindros", 2] },
>>     diferencia: {
>>       $round: [{ $subtract: ["$mpg", "$promedio_cilindros"] }, 2]
>>     }
>>   }
>> },
>> { $sort: { diferencia: -1 } },
>> { $limit: 5 }
```

```
>> { "descripcion": "Prius Hybrid", "cilindros": 4, "mpg": 55, "promedio_cilindros": 39.25, "diferencia": 15.75 },
>> { "descripcion": "Accord V6", "cilindros": 6, "mpg": 25, "promedio_cilindros": 20.5, "diferencia": 4.5 },
>> { "descripcion": "Mustang GT", "cilindros": 8, "mpg": 19, "promedio_cilindros": 17, "diferencia": 2 },
>> { "descripcion": "Civic Si 1.5t", "cilindros": 4, "mpg": 38, "promedio_cilindros": 39.25, "diferencia": -1.25 },
>> { "descripcion": "F-150 Raptor", "cilindros": 8, "mpg": 15, "promedio_cilindros": 17, "diferencia": -2 }
```

- 20. Calcular, por continente, el número de autos, cuántos tienen 8 o más cilindros y el porcentaje que representan (1 decimal).

```
PS C:\Users\elsi_> db.cars.aggregate([
>> { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },
>> { $unwind: "$maker" },
>> {
>>   $group: {
>>     _id: "$maker.continente",
>>     total_autos: { $sum: 1 },
>>     autos_8_o_mas_cilindros: {
>>       $sum: { $cond: [{ $gte: ["$cylinders", 8] }, 1, 0] }
>>     }
>>   },
>> },
>> {
>>   $project: {
>>     _id: 0,
>>     continente: "$_id",
>>     total_autos: 1,
>>     autos_8_o_mas_cilindros: 1,
>>     porcentaje: {
>>       $round: [
>>         { $multiply: [{ $divide: ["$autos_8_o_mas_cilindros", "$total_autos"] }, 100] },
>>         1
>>       ]
>>     }
>>   }
>> }
>> ]
>>
>> { "continente": "Europe", "total_autos": 2, "autos_8_o_mas_cilindros": 0, "porcentaje": 0 },
>> { "continente": "America", "total_autos": 2, "autos_8_o_mas_cilindros": 2, "porcentaje": 100 },
>> { "continente": "Asia", "total_autos": 4, "autos_8_o_mas_cilindros": 0, "porcentaje": 0 }
```

- 21. Calcular la potencia por cada 1000 lb de peso y mostrar los 10 autos con mayor valor, junto con su país.

```
PS C:\Users\elsi_> db.cars.aggregate([
>> { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },
>> { $unwind: "$maker" },
>> {
>>   $project: {
>>     _id: 0,
>>     descripcion: "$description",
>>     pais: "$maker.country",
>>     potencia_por_1000lb: {
>>       $round: [
>>         { $multiply: [{ $divide: ["$hp", "$weight_lb"] }, 1000] },
>>         2
>>       ]
>>     }
>>   }
>> },
>> { $sort: { potencia_por_1000lb: -1 } },
>> { $limit: 10 }
```

```
>> { "descripcion": "GTR 3.8t", "pais": "Japan", "potencia_por_1000lb": 144.87 },
>> { "descripcion": "Mustang GT", "pais": "USA", "potencia_por_1000lb": 118.42 },
>> { "descripcion": "Golf R 2.0t", "pais": "Germany", "potencia_por_1000lb": 95.45 },
>> { "descripcion": "Accord V6", "pais": "Japan", "potencia_por_1000lb": 81.76 },
>> { "descripcion": "F-150 Raptor", "pais": "USA", "potencia_por_1000lb": 80 },
>> { "descripcion": "Civic Si 1.5t", "pais": "Japan", "potencia_por_1000lb": 68.97 },
>> { "descripcion": "Jetta 2.0", "pais": "Germany", "potencia_por_1000lb": 50 },
>> { "descripcion": "Prius Hybrid", "pais": "Japan", "potencia_por_1000lb": 41.38 }
```

- 22. Para cada fabricante, mostrar solo los modelos cuyo nombre es distinto al nombre del fabricante, descartando a los fabricantes que no tengan ninguno.

```
PS C:\Users\elsi_> db.makers.aggregate([
>> {
>>   $project: {
>>     _id: 0,
>>     fabricante: "$name",
>>     modelos_distintos: {
>>       $filter: {
>>         input: "$models.name",
>>         as: "model_name",
>>         cond: { $ne: ["$$model_name", "$name"] }
>>       }
>>     }
>>   }
>> },
>> {
>>   $match: {
>>     $expr: { $gt: [{ $size: "$modelos_distintos" }, 0] }
>>   }
>> }
>> ])
```

```
>> { "fabricante": "Volkswagen", "modelos_distintos": ["Jetta", "Golf"] },
>> { "fabricante": "Toyota", "modelos_distintos": ["Corolla", "Prius"] },
>> { "fabricante": "Ford", "modelos_distintos": ["Mustang", "F-150"] },
>> { "fabricante": "Honda", "modelos_distintos": ["Civic", "Accord"] },
>> { "fabricante": "Nissan", "modelos_distintos": ["Sentra", "GTR", "Altima"] },
>> { "fabricante": "Audi", "modelos_distintos": ["A4"] }
```

- 23. En una sola consulta, obtener: (a) el número de autos por continente, (b) el número de autos por cilindros y (c) las estadísticas globales de mpg (mínimo, máximo y promedio).

```
PS C:\Users\elsi_> db.cars.aggregate([
>> { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },
>> { $unwind: "$maker" },
>> {
>>   $facet: {
>>     autos_por_continente: [
>>       { $group: { _id: "$maker.continent", total_autos: { $sum: 1 } } },
>>       { $project: { _id: 0, continente: "$_id", total_autos: 1 } }
>>     ],
>>     autos_por_cilindros: [
>>       { $group: { _id: "$cylinders", total_autos: { $sum: 1 } } },
>>       { $sort: { _id: 1 } },
>>       { $project: { _id: 0, cilindros: "$_id", total_autos: 1 } }
>>     ],
>>     estadisticas_globales_mpg: [
>>       {
>>         $group: {
>>           _id: null,
>>           min_mpg: { $min: "$mpg" },
>>           max_mpg: { $max: "$mpg" },
>>           avg_mpg: { $round: [{ $avg: "$mpg" }, 2] }
>>         },
>>         $project: { _id: 0 }
>>       }
>>     ]
>>   }
>> }
>> }
```

```
>> {
>>   "autos_por_continente": [
>>     { "continente": "Europe", "total_autos": 2 },
>>     { "continente": "America", "total_autos": 2 },
>>     { "continente": "Asia", "total_autos": 4 }
>>   ],
>>   "autos_por_cilindros": [
>>     { "cilindros": 4, "total_autos": 4 },
>>     { "cilindros": 6, "total_autos": 2 },
>>     { "cilindros": 8, "total_autos": 2 }
>>   ],
>>   "estadisticas_globales_mpg": [
>>     { "min_mpg": 15, "max_mpg": 55, "avg_mpg": 28.5 }
>>   ]
>> }
```

- 24. Asignar a cada auto alemán su posición por potencia dentro de su país y mostrar los 5 primeros.

```
PS C:\Users\elsi> db.cars.aggregate([
>> { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },
>> { $unwind: "$maker" },
>> { $match: { "maker.country": "Germany" } },
>> {
>>   $setWindowFields: {
>>     partitionBy: "$maker.country",
>>     sortBy: { hp: -1 },
>>     output: {
>>       posicion_potencia: { $documentNumber: {} }
>>     }
>>   },
>> { $match: { posicion_potencia: { $lte: 5 } } },
>> {
>>   $project: {
>>     _id: 0,
>>     descripcion: "$description",
>>     pais: "$maker.country",
>>     potencia: "$hp",
>>     posicion_potencia: 1
>>   }
>> },
>> { $sort: { posicion_potencia: 1 } }
>> ],
>> { "descripcion": "Golf R 2.0t", "pais": "Germany", "potencia": 315, "posicion_potencia": 1 },
>> { "descripcion": "Jetta 2.0", "pais": "Germany", "potencia": 150, "posicion_potencia": 2 }
```

- 25. Obtener para cada fabricante europeo su auto más eficiente

```
PS C:\Users\elsi> db.cars.aggregate([
>> { $lookup: { from: "makers", localField: "makerId", foreignField: "_id", as: "maker" } },
>> { $unwind: "$maker" },
>> { $match: { "maker.continent": "Europe" } },
>> { $sort: { mpg: -1 } },
>> {
>>   $group: {
>>     _id: "$maker.name",
>>     auto_mas_eficiente: { $first: "$description" },
>>     mpg: { $first: "$mpg" }
>>   }
>> },
>> {
>>   $project: {
>>     _id: 0,
>>     fabricante: "$_id",
>>     auto_mas_eficiente: 1,
>>     mpg: 1
>>   }
>> },
>> {
>>   $project: {
>>     fabricante: "$_id",
>>     auto_mas_eficiente: "$auto_mas_eficiente",
>>     mpg: "$mpg"
>>   }
>> }
>> ],
>> { "fabricante": "Volkswagen", "auto_mas_eficiente": "Jetta 2.0", "mpg": 36 }
```

Conclusión.

La realización de este proyecto nos permitió aplicar de forma práctica los conceptos de Embedding y Linking, comprobando que el diseño de esquemas en MongoDB prioriza la eficiencia en el acceso a los datos sobre la normalización estricta de los modelos relacionales. Asimismo, la construcción de las 25 consultas evidenció la gran potencia analítica del Aggregation Framework y confirmando que MongoDB ofrece tanto flexibilidad estructural como un motor robusto para procesar datos.